

PYTHON DICTIONARY

BY SUJAL SHRESTHA

PYTHON DICTIONARY

A dictionary in Python is a built-in data type that allows you to store and manipulate data in key-value pairs. It is mutable, unordered as of Python 3.6 (though it maintains insertion order in Python 3.7 and later), and can store heterogeneous data types.

Dictionary Syntax:

Creating a dictionary

```
my_dict = { "key1": "value1", "key2": "value2", "key3": "value3" }
```

Accessing values

```
print(my_dict["key1"])
```

PYTHON DICTIONARY RULES

- Store pairs of entries called items `{ 'CS' : '743-713-3350', 'UHPD' : '713-743-3333' }`.
- Each pair of entries contains a key and value.
- Key and values are separated by a colon.
- Pairs of entries are separated by commas.
- Dictionary is enclosed within curly braces.

PYTHON DICTIONARY



List

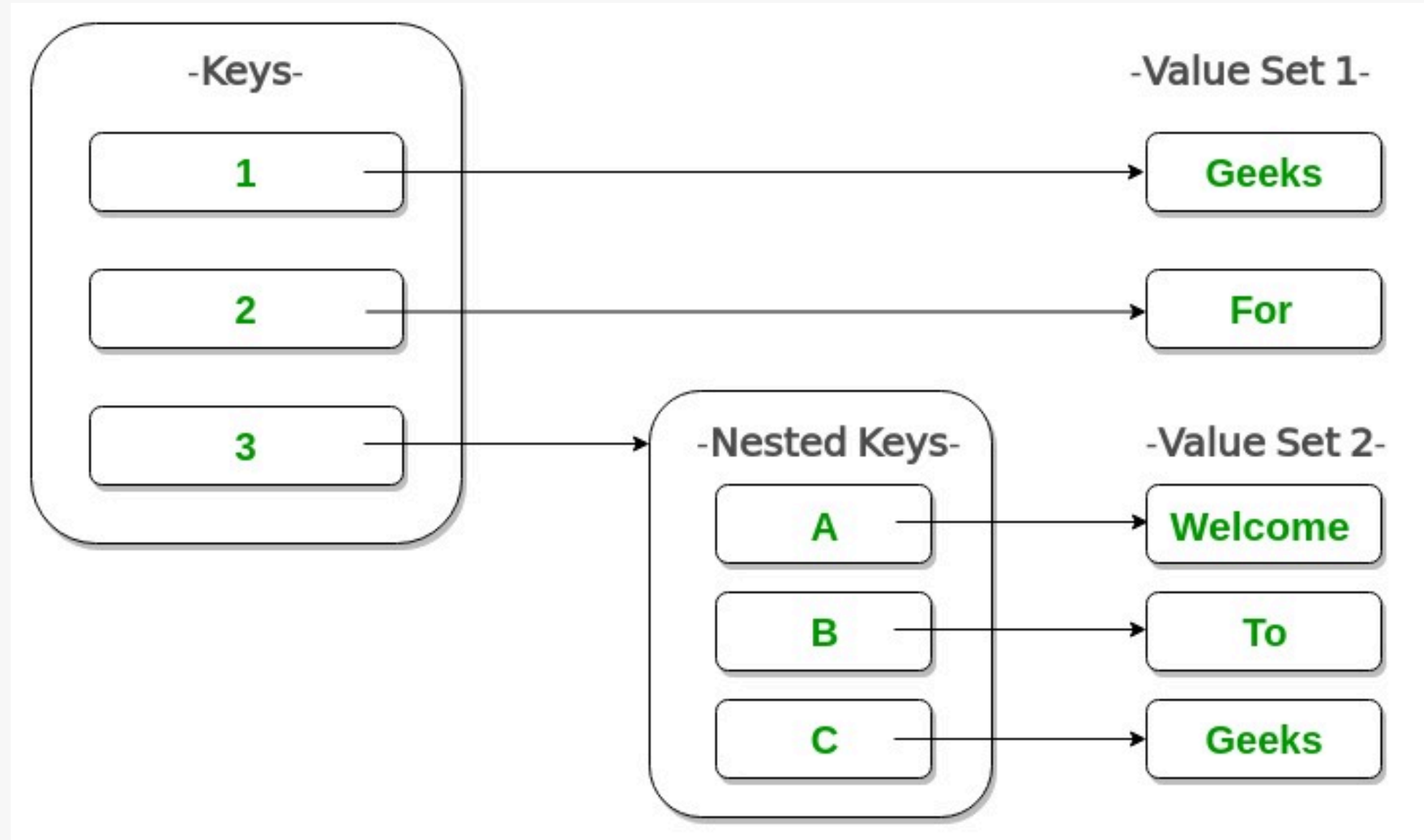
A linear collection of values that stay in order.



Dictionaries

A “bag” of values, each with its own label

PYTHON DICTIONARY



PYTHON DICTIONARY CHARACTERISTICS

a. Key-Value Structure:

Each entry in a dictionary is a key-value pair.

b. Mutable:

You can add, update, or delete items.

c. Unique Keys:

Keys in a dictionary must be unique, but values can be duplicated.

d. Key Requirements:

Keys must be immutable (e.g., strings, numbers, tuples with immutable elements).

Keys cannot be lists or other dictionaries.

PYTHON DICTIONARY OPERATIONS

a. Create a Dictionary

```
empt_dic = { }
```

```
dict_one = {"name": "Panacea", "city": "Baneshwor"}
```

b. Access Values

```
print(dict_one["name"])
```

c. Add or Update Items

```
dict_one["domain"] = "TechEd" # Adding a new key-value pair
```

```
dict_one["city"] = "Maitidevi" # Updating an existing value
```

PYTHON DICTIONARY OPERATIONS

d. Delete Items

```
del dict_one["city"] # Removes the key-value pair for "city"
```

e. Access Values

```
print(dict_one.keys()) # Output: dict_keys(['name', 'age', 'profession'])
```

```
print(dict_one.values()) # Output: dict_values(['Alice', 26, 'Engineer'])
```

f. Check if a Key Exists

```
print("name" in dict_one) # Output: True
```

```
print("salary" in dict_one) # Output: False
```


ITERATING PYTHON DICTIONARY

```
my_dict = {"name": "Minoree", "age": 25, "city": "Kathmandu"}
```

a. Iterating Over Keys

```
for key in my_dict:  
    print(key)
```

b. Iterating Over Values

```
for value in my_dict.values():  
    print(value)
```

c. Iterating Over Key-Value Pairs

```
for key, value in my_dict.items():  
    print(f"{key}: {value}")
```

ITERATING PYTHON DICTIONARY

d. Iterating Over Keys in a Sorted Order

```
for key in sorted(my_dict):  
    print(f"{key}: {my_dict[key]}")
```

e. Using Enumerate

```
for index, (key, value) in enumerate(my_dict.items()):  
    print(f"{index}: {key} -> {value}")
```

NESTED DICTONARY IN PYTHON

A nested dictionary is a dictionary inside another dictionary. It allows hierarchical storage of data, making it useful for complex data structures like JSON.

Syntax:

```
nested_dict = {  
    "person1": {"name": "Alice", "age": 25, "city": "New York"},  
    "person2": {"name": "Bob", "age": 30, "city": "Los Angeles"}  
}
```

DICITONARY METHODS

```
my_dict = {"name": "Alice", "age": 25, "city": "New York"}
```

a. `get()`

Safely access a value without raising an error if the key doesn't exist.

```
print(my_dict.get("age"))
```

```
print(my_dict.get("salary", "Not Found")) # Output: Not Found
```

b. `pop()`

Removes a specified key and returns its value.

```
print(my_dict.pop("age"))
```

c. `popitem()`

Removes and returns the last inserted key-value pair.

```
print(my_dict.popitem())
```

DICITONARY METHODS

```
my_dict = {"name": "Alice", "age": 25, "city": "New York"}
```

d. `update()`

Adds or updates multiple key-value pairs.

```
my_dict.update({"hobby": "Painting"})
```

e. `items()`

Returns a view object of all key-value pairs as tuples.

```
print(my_dict.items())
```

f. `clear()`

Removes all items from the dictionary

```
my_dict.clear()
```


THANK YOU

